

Lecture 6 Logical Design

Outline

- Relational Model
- Logical Design

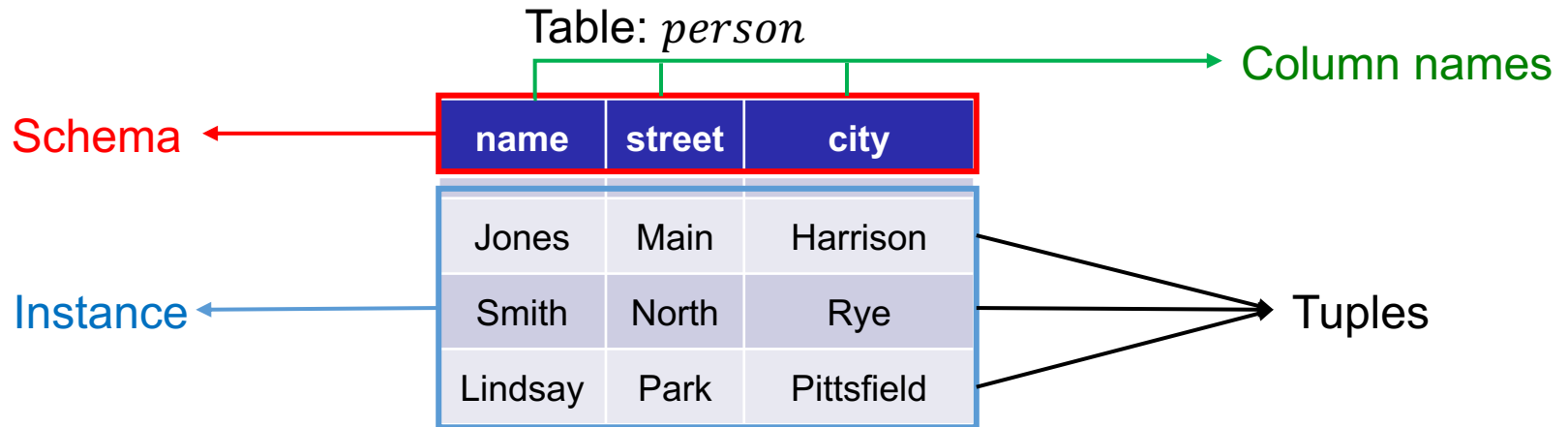
Relational Model

- After the **conceptual design**, the database designer will have an ER diagram to model a real-world problem.
- Then, the next step is the **logical design**.
- In this phase, the designer needs to specify how data is modeled in a database.
- This course introduces the ***relational model***.

Relational Model

- In the relational model, a **database** is considered as **a set of tables**. (Mathematically, a table is a **relation**.)
- Each table consists of **rows (tuples)** and **columns (attributes)**.
- Each column has a **column name**.
- Each **tuple** has a value for each column.
- The **schema** of a table is a **structural description** of tables.
- A schema is denoted as $R = (C_1, C_2, \dots, C_n)$, where R is the schema name, and $C_1, C_2, \dots, C_n$ are column names.
- If R is a schema, $r(R)$ denotes a relation on the schema R .
- An **instance** is the actual content of a table at a certain time. It consists of all rows in the table.

Example



- To denote the schema,
$$person_schema = (name, street, city)$$
- To denote the relation (or the table),
$$person(person_schema)$$
- The order of the tuples is **irrelevant**, because each relation is a set.

Relational Model

Before we proceed, you need to remember that

- The **relational model and the ER model are not parallel**. They are modeling different things and for different phases of a database design. You cannot use one to replace the other.
- The word “relation” in a relational model is different from the relationships in an ER model.
- A relation in a relational model is strictly defined in mathematics.
- An attribute in a relational model is also different from an attribute in an ER model, even they look very similar.

Keys

- The **super key** of a schema is a set of attributes that **uniquely identifies** each tuple in a relation **for any instance**.
- The **candidate key** is a **minimal** (not minimum) super key.
- The **primary key** is a candidate key that is the **most appropriate** to become the main key of the relation, never or rarely changed.

Table: student

id	name	year	gpa	phone	major
123	Richard	2	3.9	1342225648	CST
234	Grace	3	3.4	1526482319	DS
345	Bob	3	3.2	1865482164	AI

Super key: $\{id, gpa\}$, $\{phone, gender\}$, ...

Candidate key: $\{id\}$, $\{phone\}$

Primary key: $\{id\}$

- $\{year, gpa\}$ **cannot** be a super key even it can uniquely identify the tuples in this example. It cannot identify in some possible instances.

Foreign Keys

- A relational schema may have a set of attribute(s) that corresponds to the primary key of another relation. This set of attribute(s) is called a **foreign key**.
 - For example, given the following schemas
$$\begin{aligned} student &= (\underline{student_ID}, name, major, phone, year, GPA) \\ book &= (\underline{ISBN}, title, author) \\ borrow &= (\underline{student_ID}, \underline{ISBN}) \end{aligned}$$
 - The attribute *ISBN* in the schema *borrow* is a foreign key to the schema *book*, same for *student_ID*.
 - *book* is the **referenced relation**. And *borrow* is the **referencing relation**.
 - The purpose of a foreign key is to find some information across multiple tables.
 - Only values occurring in the primary key attribute of the **referenced relation** may occur in the foreign key attribute of the **referencing relation**.

Foreign Keys

- The purpose of a foreign key is to find some information across multiple tables.
- For example, to find the names of books borrowed by David,
 - one must search David's student ID in the table *borrow*;
 - then, he knows the ISBNs of books borrowed by David;
 - next, he can search for the book names using the ISBNs from the table *book*.
- To make sure the searching can always be done, only values occurring in the primary key attribute of the referenced relation may occur in the foreign key attribute of the referencing relation.
- For example, it's impossible to let *borrow* have an ISBN which does not exist in *book*.

Exercise

1. Given a relation r defined over the schema R , which of the following can always uniquely identify the tuples in r ?
 - A. any non-null attributes of R
 - B. super key of R
 - C. the first attribute in R
 - D. R itself
2. Given the following relation, list all candidate keys and super keys.

A	B	C	D
A1	B1	C1	D1
A1	B2	C2	D1
A2	B1	C2	D1

Super keys: $\{A, B\}, \{A, C\}, \{B, C\}, \{A, B, C\}, \{A, C, D\}, \{A, B, D\}, \{B, C, D\}, \{A, B, C, D\}.$

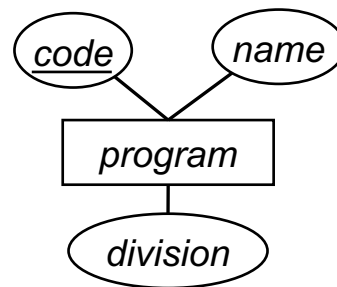
Candidate keys: $\{A, B\}, \{A, C\}, \{B, C\}.$

Logical Design

- Now, we are ready to convert ER diagrams to relational models.
- The basic ideas are as follows.
 - Build one schema for each entity set.
 - Build one schema for each relationship set only if necessary (depends on the constraints).
 - One attribute for schemas is one attribute in the ER diagram.
 - Take care of keys.
- The goals of the conversion
 - Preserve all information (from the physical world)
 - Maintain the constraints to the extent possible
 - Minimize redundancies and NULL values
 - Relational Model cannot preserve all constraints

Entity Sets

- Converting strong entity sets (without composite attributes or multivalued attributes) is straightforward.
- For example, the entity set “program”



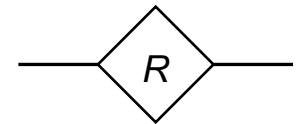
is represented by $program = (\underline{code}, name, division)$.

- For weak entity sets, the case is same as many-to-one relationship set with total participation on the many side (case 9 on page 16).
- Composite attributes and multivalued attributes are discussed at the end.

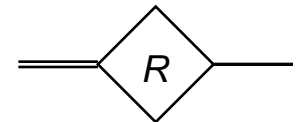
Relationship Sets

- Different constraints make the conversion of relationship sets different.
- Here we list all different cases of the constraints. (10 in total)

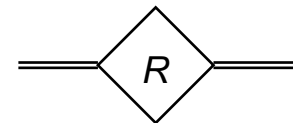
1. many-to-many without total participation



2. many-to-many with total participation on one side

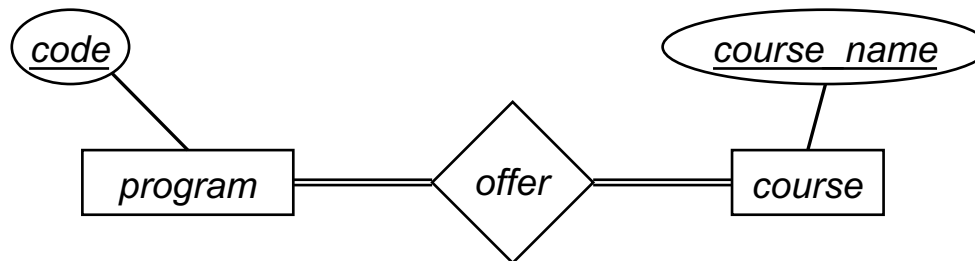


3. many-to-many with total participation on both sides



Relationship Sets

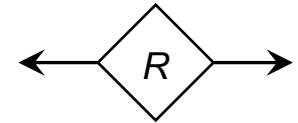
- For the above 3 cases, we create an individual schema for each relationship set.
- The column names consist of the key attributes for both entity sets and the attributes for the relationship set (if any).
- The key of the schema is the combination of the key attributes for both entity sets.
- For example, the relationship set “offer” is presented as
$$offer = (\underline{code}, \underline{course_name})$$



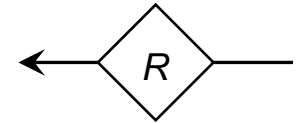
Relationship Sets

- More cases:

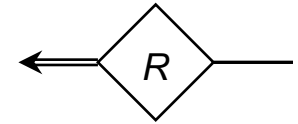
4. one-to-one without total participation



5. one-to-many without total participation



6. one-to-many with total participation on the “one” side

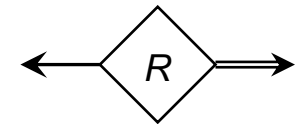


- For these three cases, the conversion is same as the previous cases.
- But the key of the schema is
 - either one of the key attributes of the entity sets (case 4);
 - the key attribute of the entity set on the “many” side (case 5 and 6).

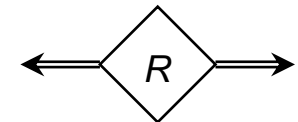
Relationship Sets

- More cases:

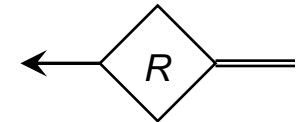
7. one-to-one with one total participation and one partial participation



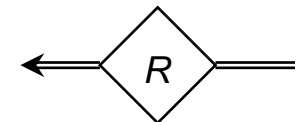
8. one-to-one with total participation on both sides



9. one-to-many with total participation on the “many” side

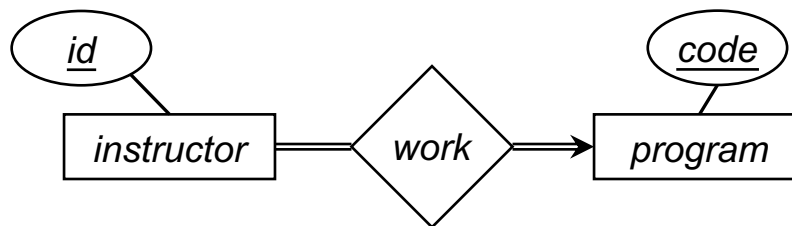


10. one-to-many with total participation on both sides



Relationship Sets

- For these four cases, there is no need to create a schema.
- We only need to add a foreign key as a reference to the schema of
 - the entity set with total participation on the “one” side (case 7 and 8)
 - the entity set with total participation on the “many” side (case 9 and 10)
- For example, the relationship set “work” associating “instructor” and “program”.

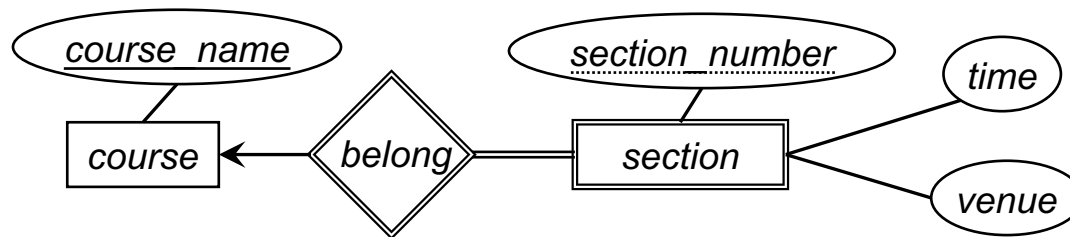


foreign key

- Because every instructor is associated with exactly one program, there is no need to create a schema for “work”.
- The schema for “instructor” is $instructor = (\underline{id}, \dots, \boxed{code})$.

Relationship Sets

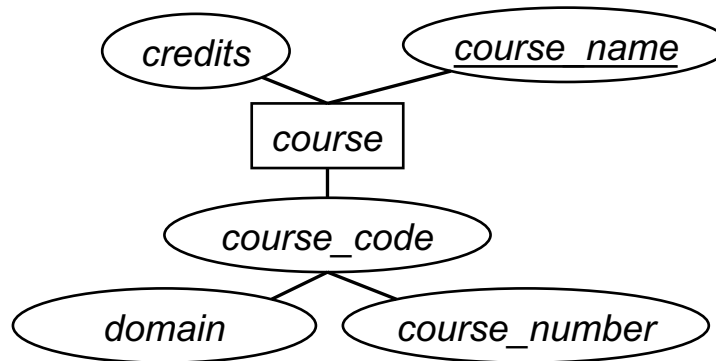
- The conversion for weak entity sets is same as case 9, one-to-many with total participation on the “many” side.
- The only difference is the key.
- For example,



- The schema for the entity set “section” is
 $section = (\underline{course_name}, section_num, time, venue)$
- There is no need to create a schema for “belong”.

Composite Attribute

- If an entity set has composite attributes, composite attributes are directly substituted by the component attributes.
- For example, the entity set “course”

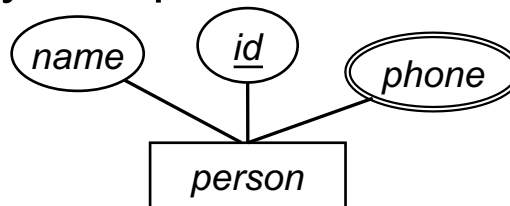


is converted into

$$course = (\underline{course_name}, credits, \\ domain, course_number)$$

Multivalued Attributes

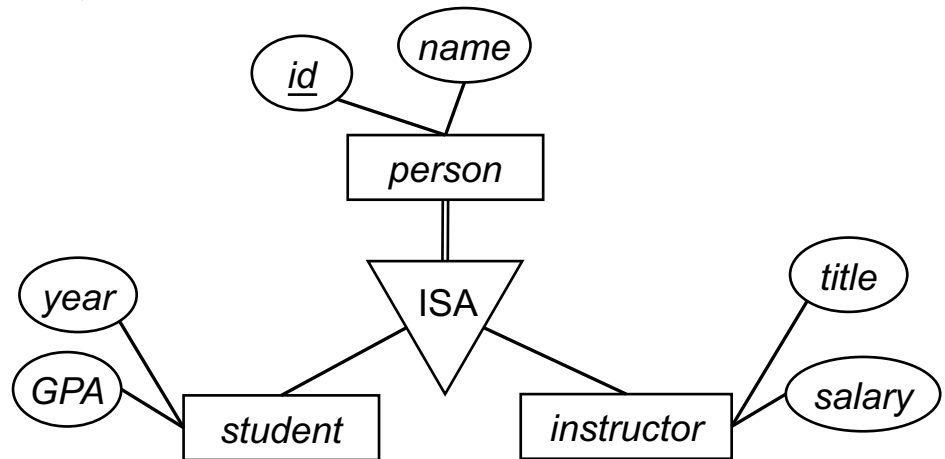
- If an entity set has a multivalued attribute, the entity set is converted into two relational schemas.
- One consists of all attributes except the multivalued attribute.
- The other one consists of the key attribute(s) and the multivalued attribute. The key of this schema is the schema itself.
- For example, the entity set “person”



is converted into $person = (\underline{id}, name)$ and $person_phone = (\underline{id}, phone)$.

ISA Relationship Sets

- If the ISA relationship set is overlapping or partial
 - the schema for the higher-level entity set is like other strong entity sets;
 - the schema for the lower-level entity set only consists of the attributes of this lower-level entity set and the key of the higher-level entity set;
 - the key of the higher-level entity set is also the key of the lower-level entity set.
- Assume that the multivalued attribute “phone” is omitted, and the ISA is not disjoint.



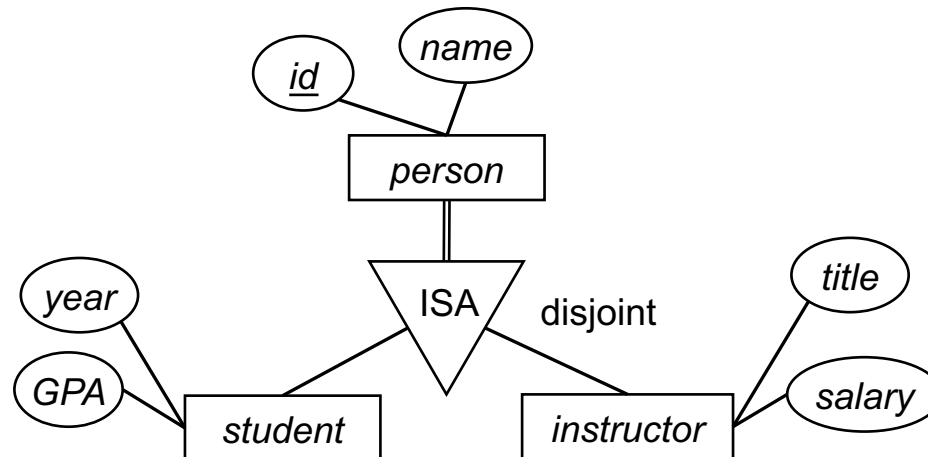
$person = (\underline{id}, name)$

$student = (\underline{id}, year, GPA)$

$instructor = (\underline{id}, title, salary)$

ISA Relationship Sets

- If the ISA relationship set is disjoint **and** total
 - no need to create a schema for the entity set on the higher level;
 - one schema is constructed for each entity set on the lower level, inheriting all attribute from the higher-level entity set.



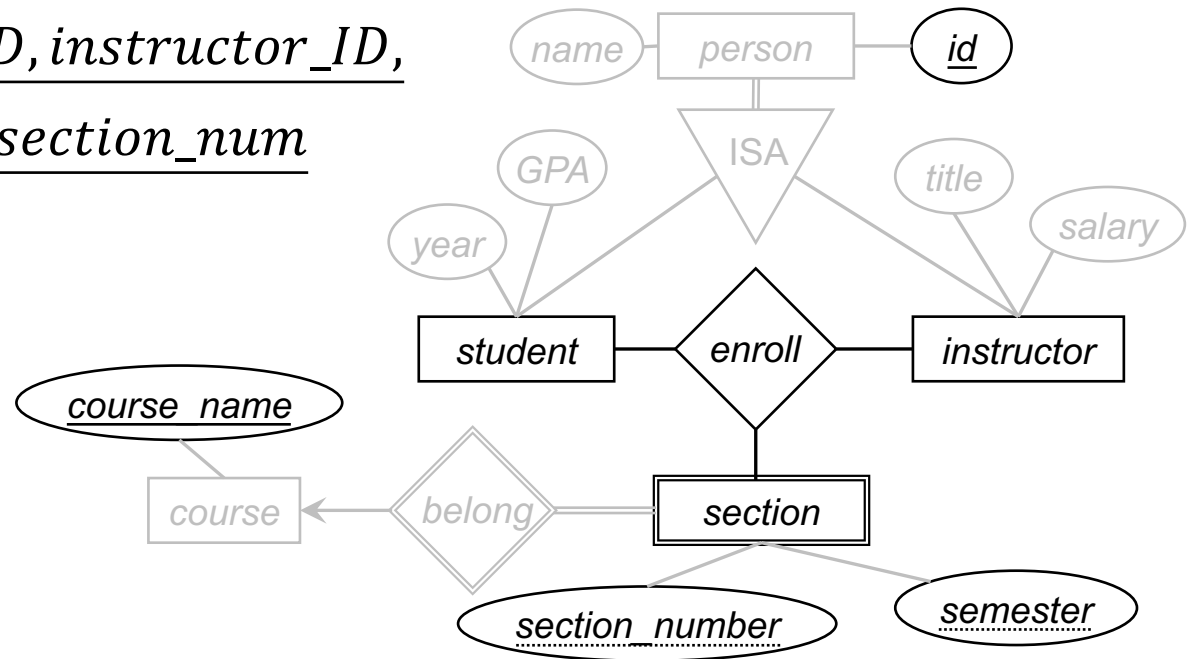
$student = (\underline{id}, name, year, GPA)$

$instructor = (\underline{id}, name, title, salary)$

Multi-ary Relationship Sets

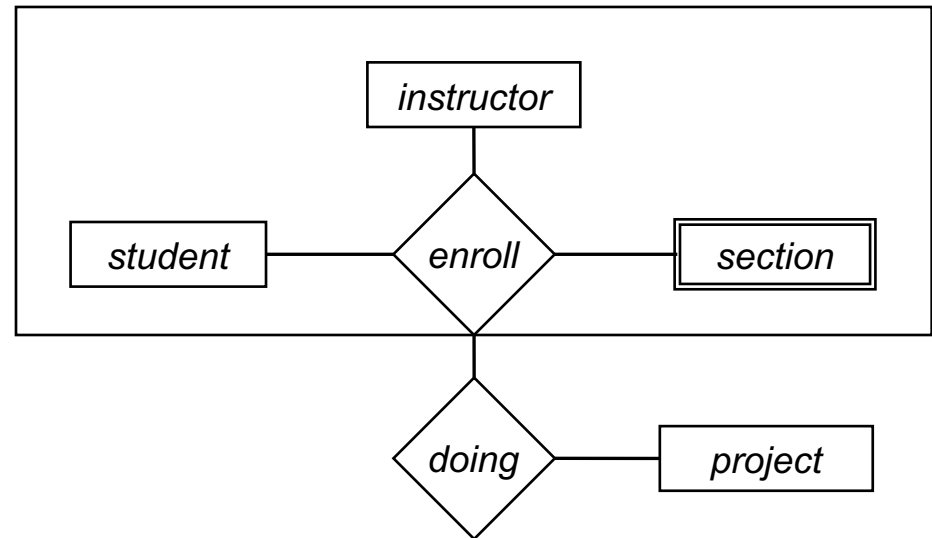
- Since we do not allow constraints on multi-ary relationship sets, the conversion is same many-to-many binary relationship sets.
- A schema contains the key attributes of all entity sets associated by the multi-ary relationship set.

$enroll = (\underline{student_ID}, \underline{instructor_ID},$
 $\underline{course_name}, \underline{section_num}$
 $\underline{semester})$



Aggregation

- Aggregations are treated same as other relationship sets.
- For example, the schema for “enroll” has the key of “instructor”, “student”, and “section”.
- Then, “doing” is a binary many-to-many relationship set. And the method for case 1 can be applied.
- Thus,



$doing = (\underline{student_ID, instructor_ID, course_name, section_number, semester, project_ID})$

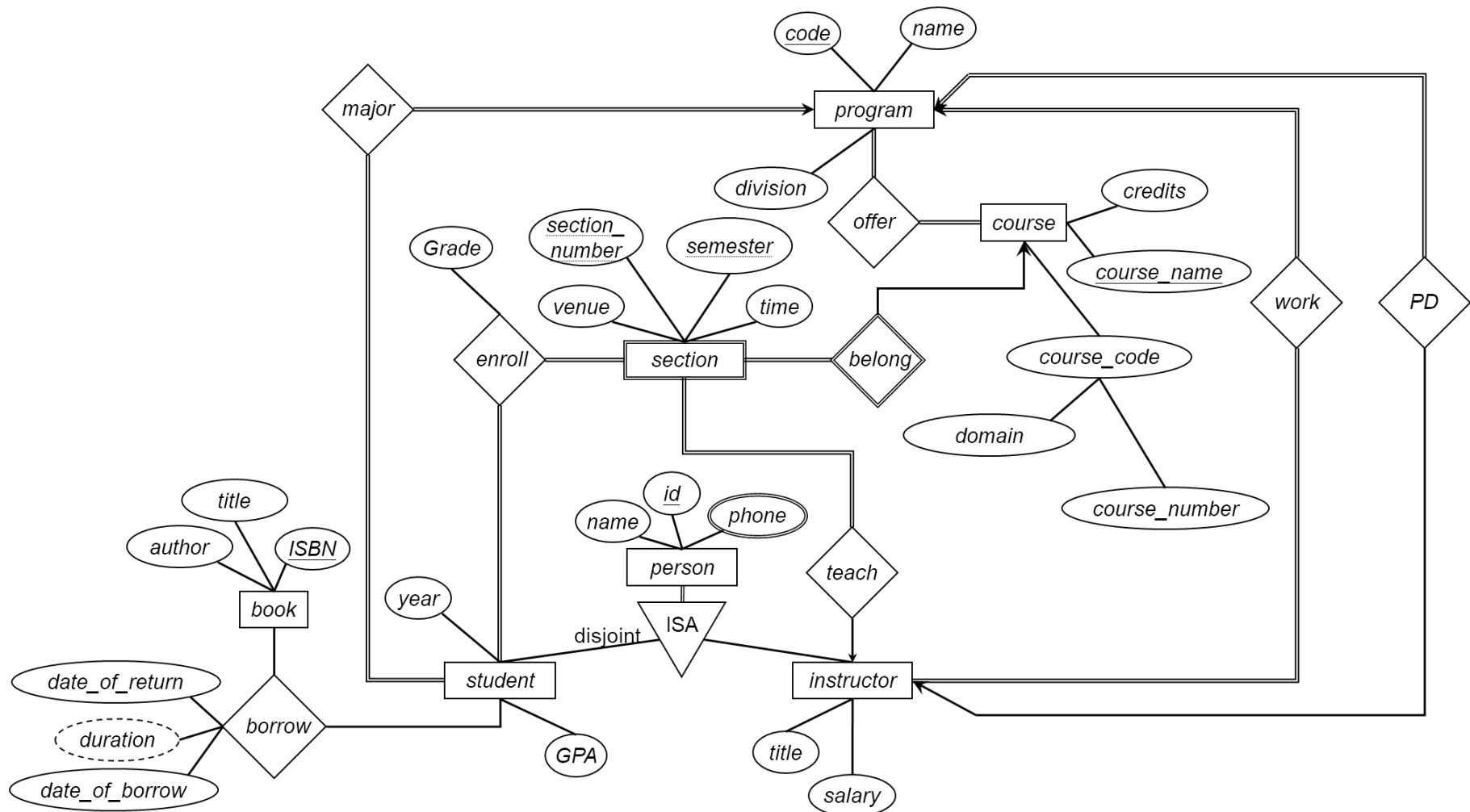
Comments

- Sometimes different entity sets may have some attributes of the same name (of course different meanings).
- It's fine to rename the attributes in the logical design to distinguish them.
- For example, we rename the *IDs* for students and instructors on the last page.

Exercises

- On page 16, if the relationship set is one of the case 7-10, there is no need to create a relational schema for the relationship set. What will happen if this method is applied to case 1-6?
- On page 20, we create two schemas for the entity sets with a multivalued attribute. What will happen if the two schema are combined as one?
- Given the ER diagram for the UIC database (on the next page), which is the outcome of the exercise of the last lecture. Please translate the diagram into relational schemas. Part of it is already done in the examples.

Exercise



End of Lecture 6